# Fine-tune SmolVLA trên LIBERO-Long bằng học tăng cường

Study · Reproduce · Analyse · Improve

Tên, lớp, email

Ngày nộp

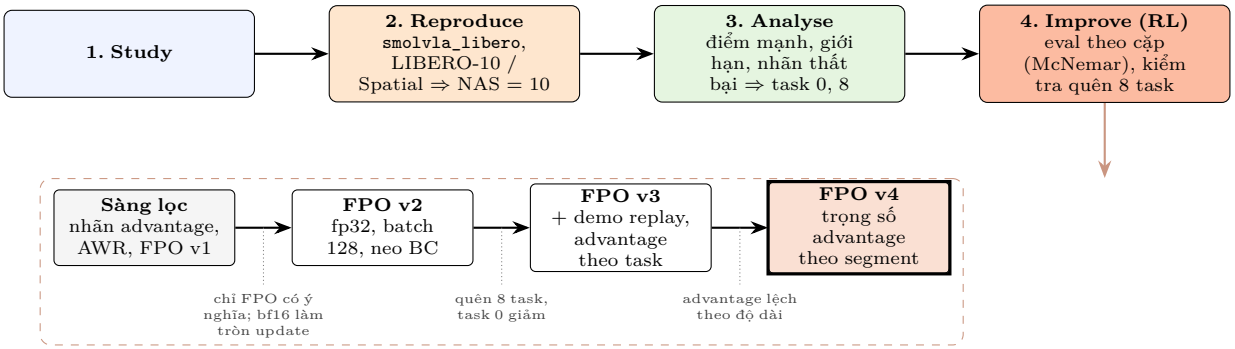
## 1 abstract

Trong báo cáo này em đã thực hiện: tái lập `lerobot/smolvla_libero` (SmolVLA 0,45B) trên LIBERO-10, phân tích thất bại, rồi fine-tune bằng RL on-policy. Đối với các mục:

**1. Reproduction:** Tái lập SmolVLA 0,45B trên LIBERO-10, đạt 72,8% (1092/1500 episode, 3 seed) với 10 action steps. Khi sử dụng 1 và 50 action steps, kết quả lần lượt là 54,6% và 47,8%.

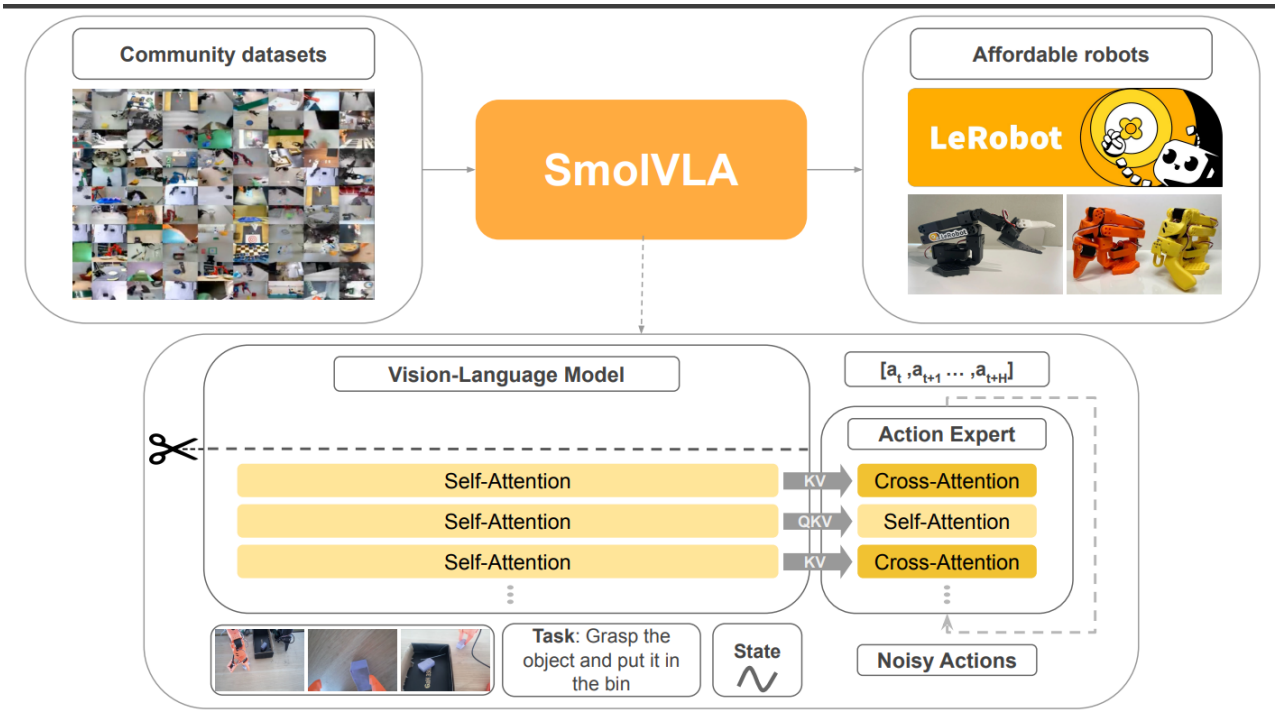
**2. Failure Analysis:** Thất bại chủ yếu tập trung ở task 0 và 8, liên quan đến việc thực hiện các bước thao tác liên tiếp, gắp và đặt vật thể.

**3. RL Improvement:** Phát triển FPO qua 4 phiên bản sau khi khảo sát RECAP, AWR và FPO. FPO v4 tăng tỷ lệ thành công trên task 0 và 8 từ 40,3% lên 46,7% ($p = 0,025$), cải thiện task 8 lên 53,3% và duy trì hiệu suất trên 8 task còn lại (78,1% so với 79,4%; $p = 0,83$). Tuy nhiên, mức thay đổi trọng số còn nhỏ (dưới 0,15%) và chưa cho thấy cải thiện có ý nghĩa thống kê trên mô hình khởi tạo chưa huấn luyện.

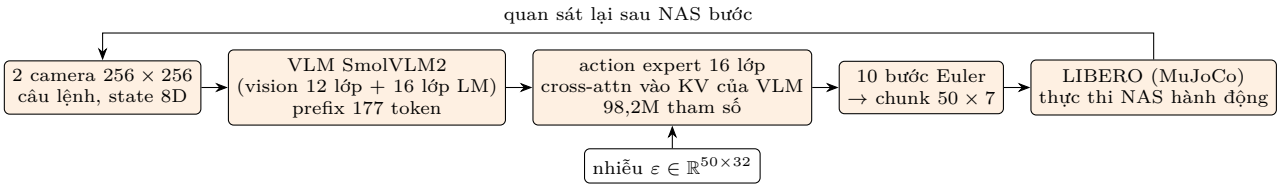



**Hình 1:** Luồng công việc. Hàng dưới: chuỗi Improve; chữ nhỏ là chẩn đoán dẫn tới phiên bản kế tiếp.

## 2 Reproduce




**Hình 2:** Kiến trúc SmolVLA.




**Hình 3:** Pipeline suy luận của `lerobot/smolvla_libero` (450,0M tham số; 96,5 ms/chunk trên RTX 5070 Ti; NAS: number action steps).

**Thiết lập.** Checkpoint `lerobot/smolvla_libero` (commit `a24f1f38`), LeRobot 0.6.2, 1×RTX 5070 Ti 16 GB, `control_mode=relative`, giới hạn 520 bước (LIBERO-10). Episode $k$ luôn dùng init state $k$; seed chỉ đổi nhiễu lấy mẫu. Thành công là khi mọi predicate BDDL đúng. Báo khoảng Wilson 95%.

| Run | Suite | NAS | Episode | Thành công (Wilson 95%) |
|---|---|---|---|---|
| B_main | LIBERO-10 | 1 | 3 seed × 10 init × 10 task | 158/300 = 52,7% (47,0–58,2) |
| D_nas1 | LIBERO-10 | 1 | 1 seed × 50 init | 273/500 = 54,6% (50,2–58,9) |
| **D_nas10** | LIBERO-10 | **10** | 3 seed × 50 init | **1092/1500 = 72,8%** (70,5–75,0) |
| D_nas50 (mặc định config) | LIBERO-10 | 50 | 1 seed × 50 init | 239/500 = 47,8% (43,5–52,2) |
| F_spatial | LIBERO-Spatial | 1 | 3 seed × 10 init | 237/300 = 79,0% (74,0–83,2) |
| F_spatial_{nas10, nas50, abs} | LIBERO-Spatial | 10 / 50 / 1† | 1 seed × 10 init | 82% / 58% / 2% |

**Bảng 1:** Kết quả tái lập. †`abs`: NAS=1 với `control_mode=absolute`.

**So với bài gốc.** Bài SmolVLA báo LIBERO-Long 71% và Spatial 90%. Inference với NAS=1, tỷ lệ thành công thu được (52,7% và 79,0%); với NAS=10 thì đạt 72,8% trên LIBERO-10. Để so sánh bản trước và sau cải thiện dùng RL, giá trị tốt nhất NAS=10 được sử dụng (để chắc chắn việc cải thiện mô hình là đến từ phần thêm RL).

**Khác biệt chính trong phần tái tạo thí nghiệm**: không train lại, dùng checkpoint công bố bởi nhóm tác giả `lerobot/smolvla_libero`; 2 suite thay vì 4; phải thêm `--rename_map` cho camera; `smolvla_base` không chạy được trên LIBERO (action 6 chiều).

# 3 Analyse

| **Điểm mạnh** | **Bằng chứng** (nguồn trong ngoặc vuông, xem chú thích) |
|---|---|
| S1 Task một bước / vật khớp nối làm tốt | [D10] tỷ lệ thành công task 3: 146/150, task 2: 138/150, task 1: 136/150, task 5: 135/150 |
| S2 Hiểu đúng câu lệnh | [D10, mắt] nhầm vật/đích chỉ ở 1/28 episode thất bại có video |
| S3 Chunk + quan sát lại sửa được sai | 22/248 episode thành công có trượt vật rồi gắp lại |
| S4 Nhỏ, nhanh | 450M tham số, 96,5 ms/chunk; train, eval, RL đều chạy trên 1 GPU 16 GB |
| **Giới hạn** | |
| L1 Với task như libero-long, thường xong bước 1 và dừng lại | [D10] hai task hai vật yếu nhất: task 0 60/150, task 8 71/150. Task 8: 75/90 thất bại mắc lỗi này, chỉ có 1 ấm trên bếp |
| L2 Kém trên các thao tác có độ phức tạp nhẹ (7 loại, Bảng 3) | 142/212 thất bại có $\geq$1 lỗi thao tác tự động, so với 5/248 thành công; gắp hỏng 111/212 so với 0/248 |
| L3 Nhạy với cách thực thi | [Reproduce, Bảng 1] NAS 1/10/50: 54,6/73,6/47,8%; `absolute` thay `relative`: 2/100 |

**Bảng 2:** Điểm mạnh và giới hạn. Nguồn: **[D10]** D_nas10, `lerobot-eval`, 10 task × 50 init × 3 seed = 150 episode/task. **[D10, mắt]** 28 thất bại của D10 có video, nhãn F0–F6 gán bằng mắt (một người). **S3, về sau của L1, L2:** arm official của eval FPO v2 (NAS=10, expert fp32): task 0, 8 × 50 init × 3 seed, 8 task khác × 20 init; 460 episode = 212 thất bại + 248 thành công; nhãn tự động theo ngưỡng (`improve/src/failure_labels.py`). Mẫu số ở các ô này là số thất bại (212, hoặc 90 của task 8) hoặc số thành công (248), không phải số episode mỗi task. Chi tiết các lỗi SmolVLA mắc phải: `docs/failure_overview.md`, `docs/analyse.md`.

**L2 chi tiết: SmolVLA thất bại vì những lỗi thao tác nào?** Bảng 3 tách L2 thành 7 loại lỗi. Dữ liệu tự động là arm `official` của eval FPO v2 (NAS=10, expert fp32): task 0, 8 × 50 init × 3 seed, cộng 8 task khác × 20 init. Tổng cộng 460 episode, trong đó 212 thất bại. Nhãn được tính từ log mô phỏng (`improve/src/failure_labels.py`, `reward_r2.py`). Một episode có thể mang nhiều nhãn. Nhãn tự động không đo hướng của vật, nên lỗi "kẹp sai tư thế" chỉ có trong nhãn bằng mắt (D10, 28 thất bại).

| **Loại lỗi** | **Cách nhận biết** | **Thất bại / thành công** | **Xảy ra ở đâu; nhãn bằng mắt D10** |
|---|---|---|---|
| L2a Gắp hỏng | nhãn A: $\geq$2 lần đóng kẹp cách vật goal <10 cm, vật chưa từng được nhấc $\geq$3 cm | **111/212** / 0/248 | task 0: 73/89 (lon alphabet soup 57, tomato sauce 17), 63/89 không nhấc được lon nào; task 8: 25/90; 8 task khác: 13/33. D10: F1 1/28 |
| L2b Trượt khỏi kẹp | sự kiện slip: mất vật $\geq$5 bước khi kẹp vẫn đóng | 90/212 / 22/248 | task 0: 58/89 thất bại so với 7/61 thành công; task 8: 17/90 so với 6/60; cả 22 episode thành công có slip đều gắp lại được |
| L2c Chạm, xô lệch, cầm vật ngoài goal | nhãn C: cầm vật không thuộc goal, hoặc chạm làm nó dịch >5 cm | 35/212 / 1/248 | task 0: 33/89, chủ yếu hộp nước cam (30); 11 episode có *cầm* vật ngoài goal, 22 chỉ chạm rồi làm dịch vật. D10: F5 1/28 (nước cam vào giỏ) |
| L2d Làm rơi vật đang mang | sự kiện drop: vật đã nhấc rơi >5 cm trong 30 bước | 13/212 / 3/248 | task 0: 6/89; task 8: 3/90; 8 task khác: 4/33 |
| L2e Làm đổ, kẹp sai tư thế | chỉ có nhãn bằng mắt F2 | không đo được | **D10: F2 9/28**, nhóm lớn nhất: ấm thứ hai của task 8 bị kẹp nằm ngang/nghiêng (3), task 0: lon bị kẹp nghiêng/đổ (2), hộp nước cam rơi (1). Nhãn tự động: 15/90 thất bại task 8 nhấc được cả hai ấm nhưng ấm 2 không lên bếp (chưa kiểm bằng video) |
| L2f Đặt gần đích, env không tính | nhãn G: vật nằm yên cách tâm đích <10 cm mà predicate vẫn sai | 13/212 / 0/248 | task 4, 6 (đặt cốc/pudding lên đĩa) 8/18; task 8 4/90. Lệch 1,2–8,6 cm, trung vị 4,3 cm (`On` cần <3 cm). D10: F4 5/28 |
| L2g Làm mất tiến độ | nhãn D: predicate goal đúng rồi sai lại | 8/212 / 1/248 | task 8: ấm đã lên bếp bị kéo ra (5/90); task 6: 3/11 |

**Bảng 3:** Các loại lỗi thao tác của SmolVLA (checkpoint official, NAS=10). Cột giữa đếm số episode có nhãn đó. Số đếm: `docs/failures/tools/l2_counts.py`; chi tiết và giới hạn: `docs/analyse.md` §2.1. Ngưỡng do người viết tự chọn; detector collision đã được kiểm bằng ảnh, slip/drop/A/G/D chưa.

Ba điểm rút ra. (1) **Gắp hỏng là lỗi thao tác chính**: có ở 111/212 thất bại và 0/248 thành công. Lỗi này thường đi cùng trượt khỏi kẹp: ở task 0, 58/89 thất bại có slip, so với 7/61 thành công. Ở task 0 lỗi xảy ra ngay ở bước con đầu tiên: robot với vào cụm hộp nơi lon alphabet soup nằm sau hộp sữa, và hay chạm hoặc cầm phải hộp nước cam. (2) **Task 8 khác task 0**: chỉ 33/90 thất bại có nhãn A/C/F/G/D. 57 thất bại còn lại chỉ có nhãn "xong một ấm" và/hoặc "đứng yên", tức là thuộc L1, hoặc là kẹp sai tư thế mà nhãn tự động không thấy. (3) **Đặt chưa đủ chính xác** (L2f) ít gặp hơn nhưng không xuất hiện ở episode thành công nào. Lỗi này tập trung ở các task đặt cốc lên đĩa.




**(a)** Task 0: lon xanh đã vào giỏ, sau đó cầm lon đỏ lơ lửng tới hết giờ (F3, L1).




**(b)** Task 8: ấm thứ hai bị kẹp nằm ngang (F2, L2).

**Hình 4:** Hai kiểu thất bại điển hình (B_main, seed 1000; 4 khung: bước 10, 1/3 episode, lúc dừng, cuối).

**Hệ quả cho Improve.** Lỗi do cách thực thi đã được sửa bằng NAS=10; L1 và L2 thì không. Vì vậy Improve tập trung vào **task 0** (hai lon vào giỏ) và **task 8** (hai ấm lên bếp), hai task có tỉ lệ thành công 30–70%, đủ đa dạng để advantage theo nhóm có tín hiệu. Reward được thiết kế từ phân tích này: +1 khi thành công, shaping theo tiến độ $\Phi$ (số predicate con đã đúng), trừ điểm khi làm rơi, trượt, va chạm và hết giờ (`improve/src/reward_r2.py`).

# 4 Improve: RL on-policy cho SmolVLA

Mục này đi theo đúng các câu hỏi của đề: giới hạn nào cần sửa và giả thuyết là gì (4.1), dùng thuật toán RL nào (4.2), reward ra sao (4.3), train trong môi trường nào (4.4), cập nhật phần nào của mô hình (4.5) và vì sao RL có thể làm policy tốt lên (4.6). Phần còn lại là cài đặt, kết quả và kiểm tra lại giả thuyết.

## 4.1 Hypothesis

**Giới hạn nào cần sửa?** Theo Mục 3, lỗi do cách thực thi (L3) đã được sửa bằng NAS=10. Còn lại hai giới hạn nằm trong chính policy:

- **L1, không chuyển sang bước con tiếp theo.** Task 8 (*put both moka pots on the stove*) có 75/90 thất bại kết thúc với đúng một ấm trên bếp.
- **L2, thao tác tinh kém.** Task 0 (*put both the alphabet soup and the tomato sauce in the basket*) có 73/89 thất bại là gắp hỏng (L2a), 58/89 có trượt khỏi kẹp (L2b), 33/89 chạm hoặc xô lệch vật ngoài goal (L2c).

Hai task này cũng là hai task yếu nhất (official: 61/150 và 60/150 ở giao thức cuối). Tỉ lệ khoảng 40% nghĩa là policy đã *đôi khi* làm được, nên RL có những rollout tốt để học theo.

**Vì sao RL có thể sửa được mà BC không sửa được?**

- BC chỉ học từ demo thành công. Trên horizon dài, nó gặp covariate shift: một lệch nhỏ ở đầu episode tích lũy thành thất bại ở cuối. Các trạng thái như "đã đặt xong ấm thứ nhất, tay đang ở vị trí lạ" hay "lon vừa trượt khỏi kẹp" hiếm khi có trong demo.
- RL học trên chính phân phối trạng thái mà policy gặp. Nó so sánh các cách xử lý khác nhau từ cùng một cảnh và đẩy policy về phía cách đi tới được mốc tiếp theo.
- Kết quả phần lớn do cảnh quyết định (D10: 289/500 cảnh thắng cả 3 seed, 69 thua cả 3, `docs/analyse.md`). Nếu cả 8 rollout của một cảnh đều thua hoặc đều thắng thì advantage theo nhóm bằng 0. Vì vậy 16 init train mỗi task được chọn trong các cảnh cho kết quả lẫn lộn ở eval gốc (`results/round2/configs/round1.yaml` dòng 15–22; tiêu chí ở `docs/WORKLOG.md`, mục D6).

**Giả thuyết.** Fine-tune on-policy bằng FPO, với reward theo tiến độ bước con và phạt sự kiện lỗi thao tác, chỉ cập nhật action expert:

**H1** tăng tỉ lệ thành công trên task 0 + 8 so với checkpoint gốc (McNemar theo cặp, $p < 0{,}05$);

**H2** tăng được là nhờ giảm thất bại kiểu L1 (xong một vật rồi dừng), vì reward theo tiến độ thưởng trực tiếp cho việc chuyển sang bước con kế tiếp;

**H3** giảm các sự kiện bị phạt của L2 (trượt khỏi kẹp, va chạm vật khác, làm rơi);

**H4** không làm giảm có ý nghĩa 8 task còn lại (quên).

H1–H4 được viết thành câu cho báo cáo này. Chỉ có ba dự đoán của v4 (Mục 4.11) là được ghi trước khi train, có timestamp.

## 4.2 RL Algorithm: FPO

**Vấn đề: $\log \pi_\theta$ không tính được.** Theo `modeling_smolvla.py`, gọi $a$ là chunk hành động, $\varepsilon \sim \mathcal{N}(0, I)$ là nhiễu, $\tau \in [0, 1]$ là thời điểm ($\tau = 1$ là nhiễu thuần):

$$x_\tau = \tau\varepsilon + (1-\tau)a, \qquad u = \varepsilon - a, \qquad \mathcal{L}_\theta(a; o) = \mathbb{E}_{\tau,\varepsilon}\big\|v_\theta(x_\tau, \tau, o) - u\big\|^2. \tag{1}$$

Khi sinh hành động, mô hình đi từ $\tau = 1$ về $\tau = 0$ qua 10 bước Euler, $x_{\tau-\Delta} = x_\tau - \Delta\, v_\theta$. Policy ngẫu nhiên chỉ nhờ nhiễu ban đầu $\varepsilon$; quá trình khử nhiễu là tất định. Vì vậy $\log \pi_\theta(a \mid o)$ không tính được trực tiếp, trong khi policy gradient và PPO đều cần nó.

**Vì sao chọn FPO?** Có ba hướng xử lý (bảng tổng hợp tài liệu: `report/literature_matrix.tex`):

- **Bỏ log-prob, dùng loss có trọng số**: AWR, hoặc gắn nhãn advantage vào điều kiện (kiểu RECAP). Rẻ, nhưng ở vòng sàng lọc (Mục 4.8) không cách nào tốt hơn checkpoint gốc.
- **Làm log-prob tính được chính xác** (Flow-SDE, $\pi_{\text{RL}}$): biến ODE thành SDE, mỗi bước Euler là một Gauss. Cách này phải sửa bộ sinh hành động và lan truyền ngược qua cả 10 bước khử nhiễu. Báo cáo này chưa thử.
- **Xấp xỉ tỉ lệ log-prob** (FPO; McAllister và cộng sự, 2025): giữ nguyên bộ sinh hành động, chỉ thay loss. Một lần lan truyền ngược trên mỗi cặp $(\tau, \varepsilon)$, vừa với một GPU 16 GB. Ở vòng sàng lọc, đây là họ duy nhất tốt hơn official có ý nghĩa.

SmolVLA có cùng kiểu kiến trúc với $\pi_0$ (VLM làm prefix, action expert flow matching, action chunk), và $\pi_0$ đã được fine-tune bằng FPO trên LIBERO ($\pi_0$-FPO, Lyu và cộng sự, 2025). Đó là lý do để kỳ vọng FPO chuyển được sang SmolVLA.

**Toán.** Với $M$ cặp $(\tau_m, \varepsilon_m)$ lưu lại lúc thu rollout và $x_m = \tau_m\varepsilon_m + (1-\tau_m)a$:

$$\hat{\mathcal{L}}_\theta(a; o) = \frac{1}{M}\sum_{m=1}^{M}\big\|v_\theta(x_m, \tau_m, o) - (\varepsilon_m - a)\big\|^2, \qquad \hat{r}(\theta) = \exp\big(c\,[\hat{\mathcal{L}}_{\theta_{\text{old}}}(a; o) - \hat{\mathcal{L}}_\theta(a; o)]\big), \tag{2}$$

$$J(\theta) = \mathbb{E}\big[\, w \,\min\big(\hat{r}\hat{A},\ \text{clip}(\hat{r}, 1\pm\epsilon)\hat{A}\big)\big] - \lambda_{\text{bc}}\,\mathcal{L}_\theta(\text{rollout thành công}) - \lambda_{\text{demo}}\,\mathcal{L}_\theta(\text{demo LIBERO-10}). \tag{3}$$

So với FPO gốc, bản cài đặt này thêm trọng số $w$ và hai số hạng neo. Cơ sở: theo bài FPO, $-\mathcal{L}_\theta$ cộng một hằng số là một ELBO của $\log \pi_\theta(a \mid o)$, nên chênh lệch loss xấp xỉ chênh lệch log-likelihood. Tại $\theta = \theta_{\text{old}}$ ta có $\hat{r} = 1$ và $\nabla_\theta(\hat{r}\hat{A}) = -c\,\hat{A}\,\nabla_\theta\hat{\mathcal{L}}_\theta$: chunk có $\hat{A} > 0$ được kéo về phía loss nhỏ hơn (dễ được sinh ra hơn), chunk có $\hat{A} < 0$ bị đẩy ra. Phần của bản cài đặt này: hệ số $c = 1/\overline{\hat{\mathcal{L}}}_{\text{old}}$ (tự chọn, không có trong bài gốc); $\epsilon = 0{,}2$; mỗi quyết định lưu 2 cặp $(\tau, \varepsilon)$, mỗi bước cập nhật dùng 1 cặp ngẫu nhiên; $w$ là trọng số theo segment (v4); hai số hạng neo BC và demo với $\lambda = 5$ để hạn chế quên (v2, v3). Cấu hình đầy đủ: `results/round2/configs/fpo_v4.yaml`.

**Advantage không cần critic.** Mỗi init được chạy 8 rollout (một nhóm). Mốc $k$ là bước mà predicate goal thứ $k$ lần đầu đúng; segment $k$ là đoạn giữa mốc $k$ và $k+1$. Giá trị $V_k$ là trung bình return của các rollout cùng nhóm tại mốc $k$:

$$A_{\text{seg}} = \sum_{t=s}^{e-1}\gamma^{t-s}r_t + \gamma^{e-s}V_{k+1} - V_k. \tag{4}$$

Một segment được chấm theo việc nó có tới được mốc kế tiếp nhanh hơn trung bình nhóm hay không, không theo những gì segment sau làm. Mỗi chunk nhận advantage của segment chứa nó.

## 4.3 Reward signal

Reward ở mỗi bước hành động $t$ (`improve/src/reward_r2.py`):

$$r_t = \underbrace{1\,[\text{thành công tại } t{+}1]}_{\text{mục tiêu}} + \underbrace{\lambda\,\big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big)}_{\text{tiến độ}} - \underbrace{\textstyle\sum_{\text{sự kiện tại } t}\text{cost}}_{\text{phạt lỗi}}, \qquad \Phi(s) = \frac{\#\text{predicate goal đúng}}{\#\text{predicate goal}}, \tag{5}$$

với $\lambda = 0{,}5$, $\gamma = 0{,}998$. Shaping dạng hiệu thế không đổi policy tối ưu (Ng và cộng sự, 1999), nên thành phần tiến độ chỉ giúp gán tín dụng, không đổi mục tiêu. Mỗi thành phần nhắm vào một lỗi đã thấy ở Mục 3:

| Thành phần | Trọng số | Khi nào kích hoạt | Nhắm vào |
|---|---|---|---|
| thành công | $+1$ | mọi predicate BDDL đúng | mục tiêu chính |
| tiến độ $\Phi$ | $\lambda = 0{,}5$ | một predicate goal chuyển sai $\rightarrow$ đúng (hoặc ngược lại thì trừ) | L1, L2g |
| slip | $-0{,}1$ | vật goal đang được kẹp tuột ra $\geq 5$ bước khi kẹp vẫn đóng | L2b |
| collision | $-0{,}1$ | vật ngoài goal bị chạm rồi dịch $> 5\,$cm (một lần mỗi vật) | L2c |
| drop | $-0{,}2$ | vật goal đã nhấc rơi $> 5\,$cm trong 30 bước | L2d |
| timeout | $-0{,}2$ | hết 520 bước mà chưa thành công | L1 (dừng, lơ lửng) |

**Bảng 4:** Thành phần reward. Gắp hỏng (L2a) và kẹp sai tư thế (L2e) không có phạt riêng; chúng chỉ bị phạt gián tiếp qua việc không có tiến độ $\Phi$ và timeout. Một lỗ hổng đã biết: vật goal bị tay đang mở đẩy đổ không kích hoạt sự kiện nào.

## 4.4 Training environment

- **Mô phỏng:** LIBERO-10 (MuJoCo), cùng cấu hình như Reproduce: `control_mode=relative`, NAS=10, tối đa 520 bước. Quan sát gồm 2 camera $256 \times 256$, state 8D và câu lệnh. Mỗi quyết định là một chunk $50 \times 7$, trong đó 10 hành động đầu được thực thi.
- **Task train:** task 0 và task 8, mỗi task 16 init (danh sách trong `fpo_v4.yaml`). 34 init/task còn lại không dùng để train và được giữ làm held-out.
- **Một vòng:** 16 init $\times$ 8 rollout $\times$ 2 task $= 256$ episode thu bằng policy hiện tại; mỗi episode có log predicate BDDL và vị trí vật ở từng bước để tính $\Phi$ và các sự kiện. Sau đó tối đa 3 epoch cập nhật, dừng sớm khi clip_frac $> 0{,}2$. Tổng 4 vòng, khoảng 3 giờ mỗi phiên bản trên 1 GPU 16 GB.
- **Seed thu rollout** giống nhau giữa các phiên bản, nên các phiên bản chỉ khác nhau ở thuật toán.

## 4.5 Thành phần nào được cập nhật?

- **Đóng băng:** vision encoder và toàn bộ VLM SmolVLM2 (`freeze_vision_encoder`, `train_expert_only`, `improve/scripts/fpo_train.py`).
- **Được train:** action expert (16 lớp) và 5 lớp chiếu (`state_proj`, `action_in_proj`, `action_out_proj`, `action_time_mlp_in`, `action_time_mlp_out`). Tổng cộng 155/500 tensor, 99,9M trên 450M tham số (`train_info.json`). Trọng số chính giữ ở fp32 (từ v2).

Có ba lý do. (1) Hai giới hạn cần sửa nằm ở *cách hành động*, không nằm ở việc hiểu câu lệnh (S2), và chính action expert sinh ra chunk. (2) Đây cũng là phần mà recipe fine-tune mặc định của SmolVLA train (`train_expert_only=True` là giá trị mặc định trong `smolvlm_with_expert.py`), nên RL giữ đúng cách checkpoint đã được train. (3) Giữ VLM nguyên vẹn thì giảm được rủi ro quên trên 8 task còn lại. Cái giá: RL không sửa được lỗi do nhận thức. Ví dụ, nếu VLM không nhận ra lon alphabet soup bị che sau hộp sữa thì RL trên expert không giúp được.

## 4.6 RL cải thiện policy như thế nào?

- **Cơ chế.** Gradient $-c\hat{A}\nabla_\theta \hat{\mathcal{L}}_\theta$ chỉnh trường vận tốc $v_\theta$ để cùng một quan sát $o$ thì chunk có advantage dương dễ được sinh ra hơn từ nhiễu. Policy không học hành vi mới từ đầu; nó dồn xác suất về những chunk tốt mà nó đã tự sinh ra được.
- **Với L1.** Trong nhóm 8 rollout cùng cảnh, rollout nào chuyển từ ấm thứ nhất sang ấm thứ hai nhanh hơn trung bình thì segment đó có $\hat{A} > 0$; rollout đứng yên sau ấm thứ nhất thì segment cuối dài, chịu timeout và có $\hat{A} < 0$. Đây chính là tín hiệu BC không có.
- **Với L2.** Segment có slip, collision hoặc drop bị trừ return. Segment gắp thành công tới được mốc tiếp theo nên có $\hat{A} > 0$. Gắp hỏng không có phạt riêng, nên L2a chỉ được sửa gián tiếp.
- **Giữ cập nhật nhỏ.** Clip, dừng sớm, neo BC và demo replay giữ policy gần $\theta_0$ để không quên 8 task còn lại. Cái giá là bước cập nhật rất nhỏ (Mục 4.12).

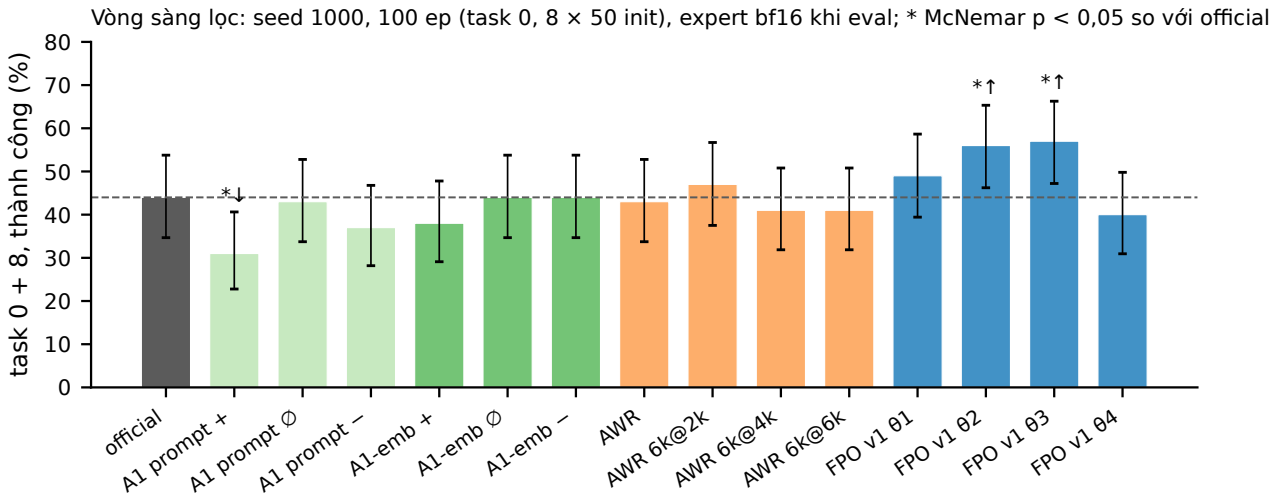
## 4.7 Implement: giao thức

- **Train:** như Mục 4.4: 4 vòng, mỗi vòng 256 rollout on-policy, advantage theo segment.
- **Đánh giá theo cặp:** task 0, 8 $\times$ 50 init $\times$ seed 3000/4000/5000 (300 cặp/arm; 34 init/task chưa dùng để train = *held-out*); quên: 8 task còn lại $\times$ 20 init, seed 1000. Cặp = cùng (seed, task, init); McNemar chính xác hai phía; không hiệu chỉnh nhiều phép so sánh.

- **Chọn checkpoint ghi trước:** giữ $\theta_4$ trừ khi tỉ lệ thành công lúc thu rollout giảm $> 15$ điểm giữa hai vòng; không chọn theo eval.

## 4.8 Vòng sàng lọc

Ba họ phương pháp được so trên cùng giao thức (seed 1000, 100 episode, expert bf16 khi eval): **A1** gắn nhãn advantage positive/negative vào prompt hoặc qua embedding học được (kiểu RECAP), **A2** AWR (loss flow có trọng số advantage), và **FPO v1** (phương trình 3). Chỉ FPO v1 $\theta_2$, $\theta_3$ tốt hơn official có ý nghĩa (56% và 57% so với 44%, $p = 0{,}01$); A1 prompt positive *kém hơn* ($p = 0{,}04$); nhãn positive và negative không tách nhau (chênh $-6$ điểm, $p = 0{,}29$). Nguyên nhân khả dĩ cho AWR/A1: action expert được nạp ở bf16, bước cập nhật nhỏ hơn độ phân giải bf16 (sau 6000 bước chỉ 5,5% phần tử của expert thay đổi). Hai phát hiện này dẫn tới FPO v2.




**Hình 5:** Vòng sàng lọc (nguồn: `results/round2/episodes.csv`). Mũi tên ↑ / ↓: tốt hơn/kém hơn official có ý nghĩa ($p < 0{,}05$).

## 4.9 Chuỗi phiên bản FPO

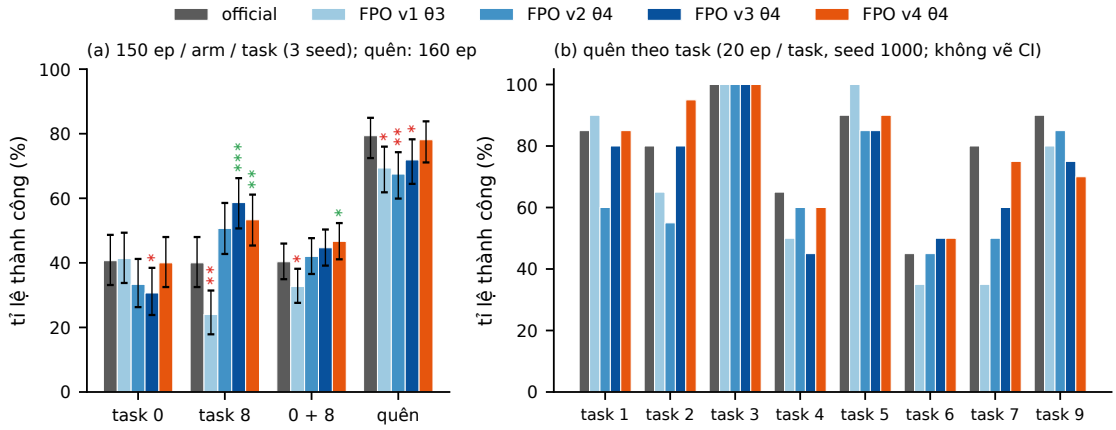
| Bản | Thay đổi so với bản trước | Lý do (chẩn đoán) | Bước grad. |
|---|---|---|---|
| v1 | FPO, batch 32, expert bf16 | — | 262 |
| v2 | trọng số chính fp32, eval expert fp32; batch 128; neo BC $\lambda = 5$ trên rollout thành công; dừng sớm khi clip_frac $> 0{,}2$; tối đa 3 epoch | cập nhật bị làm tròn ở bf16; tỉ lệ $\hat{r}$ nhiễu ở batch 32 | 84 |
| v3 | + demo replay LIBERO-10 (32 mẫu/bước, $\lambda = 5$, cả 10 task); chuẩn hoá advantage theo task; batch cân bằng 64/64 | v2 làm quên 8 task ($p = 0{,}003$) và giảm task 0 | 402 |
| v4 | trọng số $w = 1/n_{\text{decision}}$(segment), không trừ mean | ở v3, 44–53% quyết định của rollout *thất bại* ở task 0 có $\hat{A} > 0$ (Hình 8) | 228 |

**Bảng 5:** Mỗi phiên bản giữ nguyên mọi thứ khác ( 0, task, init, 4 vòng, 256 rollout/vòng, reward, seed thu rollout, quy tắc chọn). Cấu hình: `results/round2/configs/fpo\{,_v2,_v3,_v4\}.yaml`.

## 4.10 Kết quả

| | official | FPO v1 $\theta_3$ | FPO v2 $\theta_4$ | FPO v3 $\theta_4$ | **FPO v4 $\theta_4$** |
|---|---|---|---|---|---|
| task 0 (150) | 61 | 62 | 50 | 46 ▼ | **60** |
| task 8 (150) | 60 | 36 ▼ | 76 | 88 ▲ | **80** ▲ |
| tổng 0+8 (300) | 40,3% | 32,7% ▼ | 42,0% | 44,7% | **46,7%** ▲ |
| v4 vs official (chỉ v4 / chỉ off.) | | | | | 42/23, $p = 0{,}025$ |
| held-out 0+8 (204) | 84 | 66 ▼ | 86 | 87 | 91 |
| quên, 8 task (160) | 127 | 111 ▼ | 108 ▼ | 115 ▼ | **125** |
| v4 vs official | | | | | 10/12, $p = 0{,}83$ |

**Bảng 6:** Giao thức cuối (expert fp32). ▲/▼: khác official có ý nghĩa ($p < 0{,}05$, McNemar theo cặp). v4 so với v3: task 0 21/7 ($p = 0{,}013$), task 8 17/25 ($p = 0{,}28$), quên 17/7 ($p = 0{,}064$). Nguồn: `results/round2/fpo_v4/pooled.md`; v1 tính từ `improve/outputs/fpo_v2/eval_s*/fpo_v1_it3`.




Sao: McNemar chính xác so với official (cặp cùng seed, task, init); xanh = tốt hơn, đỏ = kém hơn; * p<0,05, ** p<0,01, *** p<0,001; không hiệu chỉnh nhiều phép so sánh.

**Hình 6:** So sánh cuối. (a) Thanh lỗi Wilson 95%; sao: McNemar so với official. (b) Quên theo task (20 episode/task).

**Dự đoán ghi trước (v4).** Trước khi train v4, chúng tôi ghi (có timestamp, `docs/WORKLOG.md`) giả thuyết "task 0 giảm ở v2/v3 một phần do advantage lệch theo độ dài" và ba dự đoán với tiêu chí cố định: (a) task 0 không còn kém official ($p > 0{,}05$) và cao hơn v3: **đúng** (60 vs 61, $p = 1{,}0$; $60 > 46$); (b) task 8 vẫn hơn official có ý nghĩa: **đúng** ($p = 0{,}002$); (c) quên không tệ hơn v3: **đúng** ($125 \geq 115$). Trên init held-out, v4 chưa khác official có ý nghĩa (91 vs 84/204, $p = 0{,}32$; task 8 held-out 53 vs 45/102, $p = 0{,}13$).

## 4.11 Kiểm tra giả thuyết H1–H4

Kiểu lỗi được so theo cặp giữa official và v4 trên cùng 300 (seed, task, init) (script `improve_error_shift.py` trong `docs/failures/tools/`; dữ liệu `results/round2/fpo_v4/seed*/episodes.csv`). "Xong một nửa" nghĩa là ở cuối episode đúng một trong hai vật goal đã ở đích.

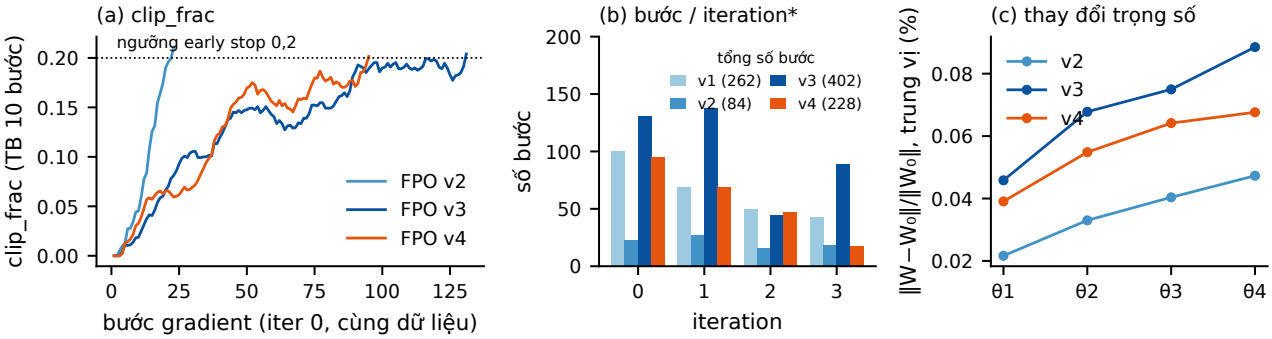
| Kiểu lỗi (số episode) | Task | official | v4 | chỉ official / chỉ v4 | $p$ |
|---|---|---|---|---|---|
| thất bại, xong một nửa (L1) | 8 | 75 | **53** | 35 / 13 | **0,002** |
| | 0 | 21 | 16 | 13 / 8 | 0,38 |
| thất bại, chưa xong vật nào | 0 + 8 | 83 | 91 | 13 / 21 | 0,23 |
| có slip (L2b) | 0 + 8 | 88 | 76 | 44 / 32 | 0,21 |
| có collision (L2c) | 0 + 8 | 32 | 34 | 14 / 16 | 0,86 |
| có drop (L2d) | 0 + 8 | 11 | 18 | 10 / 17 | 0,25 |
| thất bại, nhãn chính gắp hỏng (L2a) | 0 + 8 | 90 | 81 | 42 / 33 | 0,36 |

**Bảng 7:** Kiểu lỗi của official và FPO v4 (task 0, 8 × 50 init × 3 seed; McNemar chính xác theo cặp; sự kiện và nhãn là tự động, chưa kiểm bằng video).

- **H1 (tăng thành công): được ủng hộ.** 46,7% so với 40,3%, $p = 0{,}025$ (Bảng 6). Mức tăng nằm hoàn toàn ở task 8 (80 so với 60); task 0 giữ nguyên (60 so với 61). Trên init held-out, mức tăng chưa có ý nghĩa thống kê.
- **H2 (tăng nhờ giảm L1): được ủng hộ ở task 8.** Số thất bại "xong một ấm rồi dừng" giảm từ 75 xuống 53 ($p = 0{,}002$), vừa khớp với 20 episode thành công tăng thêm. Số thất bại chưa xong ấm nào không đổi (15 so với 17). Như vậy RL đã dạy policy chuyển sang ấm thứ hai, đúng điều reward tiến độ thưởng.
- **H3 (giảm sự kiện L2): không được ủng hộ.** Slip giảm nhẹ (88 xuống 76), collision giữ nguyên, drop tăng nhẹ, và đều không có ý nghĩa thống kê. Gắp hỏng ở task 0 cũng không giảm rõ. Lời giải thích khả dĩ, chưa kiểm chứng: phạt 0,1–0,2 nhỏ so với +1 thành công; gắp hỏng không có phạt riêng; và bước cập nhật rất nhỏ (Mục 4.12).
- **H4 (không quên): được ủng hộ.** 125 so với 127/160 trên 8 task còn lại, $p = 0{,}83$. Các phiên bản trước v4 đều quên có ý nghĩa.

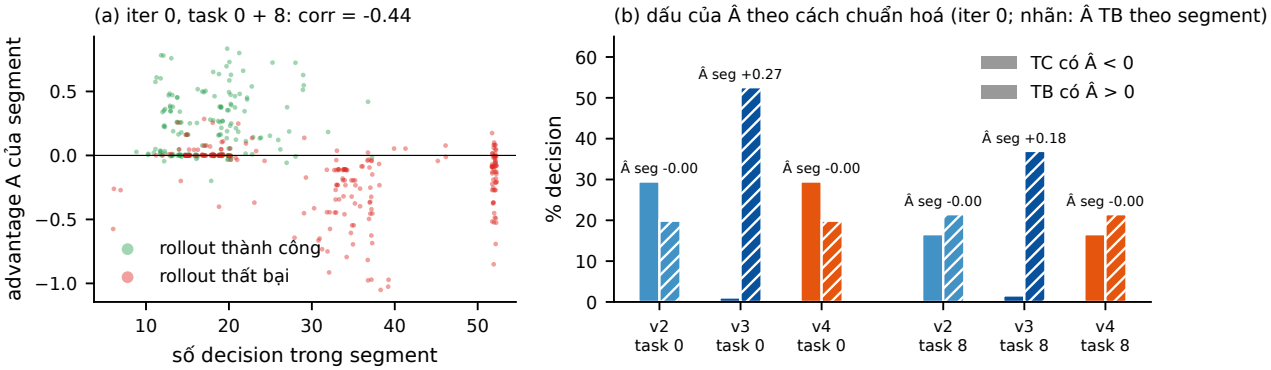
Tóm lại, RL trên action expert đã sửa được một phần L1 (chuyển bước con ở task 8) nhưng chưa sửa được L2 (độ chính xác khi gắp). Đây là lý do task 0, nơi lỗi chủ yếu là L2, không tăng.

## 4.12 Chẩn đoán




* v1: batch 32; v2–v4: batch 128 (v3, v4: 64 / task).

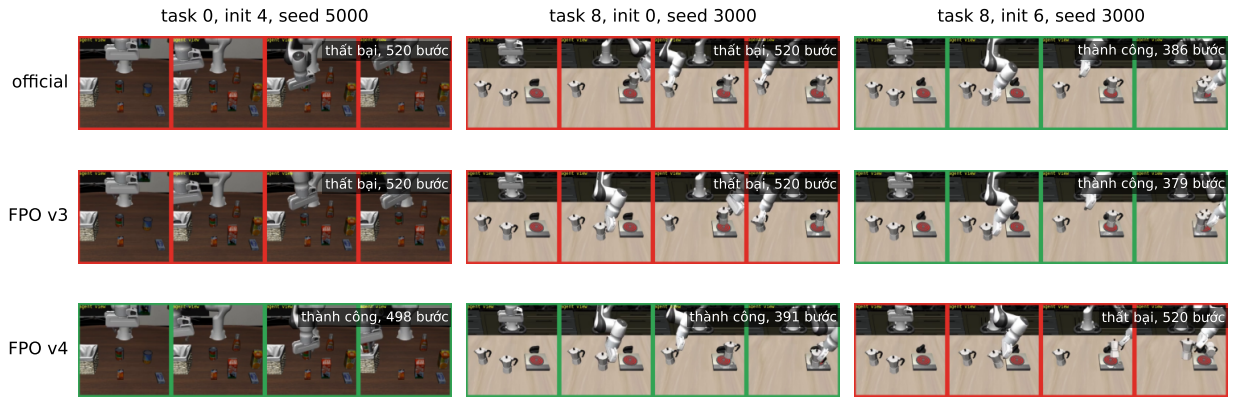
**Hình 7:** (a) Trên cùng dữ liệu iter 0, clip_frac của v2 vượt ngưỡng sau 23 bước; v3/v4 tăng chậm hơn nên chạy lâu hơn. (b) Mọi vòng đều dừng sớm. (c) Trọng số chỉ thay đổi 0,05–0,09% (trung vị theo tensor) so với $\theta_0$.




**Hình 8:** (a) Rollout thất bại luôn chạy hết 520 bước nên segment cuối dài và có advantage âm (tương quan −0,44): trung bình theo quyết định bị kéo âm dù trung bình theo segment ≈ 0. (b) v3 trừ mean theo quyết định nên 52,6% quyết định của rollout thất bại ở task 0 có $\hat{A} > 0$; v4 giữ dấu advantage gốc và trung bình theo segment ≈ 0.

- **Update rất nhỏ.** Tỉ lệ FPO được scale bởi $c = 1/\overline{\mathcal{L}}_{\text{old}} \approx 24{,}6$; một mẫu bị clip khi loss đổi $> 0{,}0074$ (18% loss trung bình), nên mọi vòng dừng sớm sau 16–138 bước với lr $10^{-6}$. Trọng số chỉ dịch dưới 0,15% (Hình 7c). Nạp checkpoint cho eval đã được kiểm tra: 155/155 tensor đã train khớp từng bit ở fp32.
- **Advantage thô đúng** (mọi quyết định khớp đúng rollout và segment; trung bình theo segment $|\bar{A}| \leq 0{,}0023$); độ lệch nằm ở *trọng số theo quyết định* (Hình 8).
- **Kiểu thất bại** (`subtask_completion` ở cuối episode, `results/round2/fpo_v4/seed*/episodes.csv`): ở task 8, số thất bại chưa có ấm nào trên bếp là 15 (official), 52 (v2), 14 (v3), 17 (v4). v2 thường hỏng ngay ấm thứ nhất; v3 và v4 thì không. Thất bại của v4 chủ yếu là xong một ấm (53/70).

### 4.13 Ví dụ định tính




**Hình 9:** Cùng (seed, task, init), replay chính xác từ eval gốc (kiểm tra `replay_check_*.csv`): official, v3, v4. Mỗi dải: 4 khung đều nhau của camera agent; viền xanh = thành công, đỏ = thất bại. Video: `results/videos/3_v4_paired/`.

**[Cần bạn điền / kiểm tra]** Mô tả ngắn hành vi khác nhau giữa v4 và official/v3 sau khi xem video `results/videos/3_v4_paired/`.

## 5 Hạn chế và kết luận

- Mỗi phiên bản chỉ train một lần (một seed train); khác biệt giữa các phiên bản có thể một phần do ngẫu nhiên khi train.
- p-value theo tập con không hiệu chỉnh; mức tăng trên init held-out chưa có ý nghĩa.
- Cơ chế khiến v3 chạy nhiều bước hơn v2 (trừ mean hay số hạng demo) chưa được tách bằng ablation.
- Nhãn thất bại A–G là tự động; nhãn F0–F6 do một người gán từ khung ảnh.
- Chỉ 2/10 task được train; chưa thử Flow-SDE (log-prob chính xác).

**Hướng tiếp theo: DSRL** (Wagenmaker và cộng sự, 2025). Giữ nguyên SmolVLA và học một policy nhỏ chọn nhiễu ban đầu $\varepsilon$ của flow matching. Hướng này không cần gradient qua VLA. Kỳ vọng (chưa kiểm chứng): không sửa trọng số SmolVLA nên không gây quên các task khác; đổi lại, chỉ chọn được trong những hành vi SmolVLA đã tạo ra được từ nhiễu. Báo cáo này chưa đánh giá DSRL.

**[Cần bạn điền / kiểm tra]** Kết luận và hướng tiếp theo (ví dụ: scale tỉ lệ FPO

## Tài liệu tham khảo


- Shukor và cộng sự (2025). SmolVLA. arXiv:2506.01844.
- McAllister và cộng sự (2025). Flow Matching Policy Gradients (FPO). arXiv:2507.21053.
- Lyu và cộng sự (2025). Reinforcement Fine-Tuning of Flow-Matching Policies for VLA Models ($\pi_0$-FPO). arXiv:2510.09976.
- Ng, Harada, Russell (1999). Policy invariance under reward transformations: theory and application to reward shaping. ICML.
- Chen và cộng sự (2025). $\pi_{\text{RL}}$: Online RL fine-tuning for flow-based VLA. arXiv:2510.25889.
- Physical Intelligence (2025). $\pi^*_{0.6}$ / RECAP. arXiv:2511.14759.
- SimpleVLA-RL (2025). arXiv:2509.09674.
- RIPT-VLA (2025). arXiv:2505.17016.
- VLA-RL (2025). arXiv:2505.18719.
- Wagenmaker và cộng sự (2025). Steering diffusion policy with latent space RL (DSRL). arXiv:2506.15799.
- Liu và cộng sự (2023). LIBERO. NeurIPS Datasets and Benchmarks.


**[Cần bạn điền / kiểm tra]** Kiểm tra tên tác giả, năm và venue của từng tài liệu.